

STAR RX72N - pwm_test Bench Test
1.0.0

Generated by Doxygen 1.16.1

1 Directory Hierarchy	1
1.1 Directories	1
2 File Index	3
2.1 File List	3
3 Directory Documentation	5
3.1 pwm_test Directory Reference	5
4 File Documentation	7
4.1 pwm_test/clock.c File Reference	7
4.1.1 Detailed Description	8
4.1.2 Enumeration Type Documentation	8
4.1.2.1 clock_byte_constants_t	8
4.1.2.2 clock_constants_word_t	9
4.1.2.3 clock_extra_addrs_t	9
4.1.2.4 clock_sckcr_t	9
4.1.3 Function Documentation	10
4.1.3.1 clock_init()	10
4.1.3.2 packcr_reg()	10
4.2 clock.c	11
4.3 pwm_test/linker.ld File Reference	13
4.4 linker.ld	13
4.5 pwm_test/main.c File Reference	14
4.5.1 Detailed Description	15
4.5.2 Macro Definition Documentation	16
4.5.2.1 GPTW0_BASE	16
4.5.2.2 GTBER	16
4.5.2.3 GTCCRA	17
4.5.2.4 GTCCRB	17
4.5.2.5 GTCNT	17
4.5.2.6 GTCR	17
4.5.2.7 GTINTAD	17
4.5.2.8 GTIOR	17
4.5.2.9 GTPBR	18
4.5.2.10 GTPR	18
4.5.2.11 GTST	18
4.5.2.12 GTUDDTYC	18
4.5.2.13 GTWP	18
4.5.2.14 MSTPCRA	18
4.5.2.15 P17PFS	19
4.5.2.16 P23PFS	19
4.5.2.17 PA7_MASK	19
4.5.2.18 PORT1_PDR	19

4.5.2.19	PORT1_PMR	19
4.5.2.20	PORT1_PODR	19
4.5.2.21	PORT2_PDR	20
4.5.2.22	PORT2_PMR	20
4.5.2.23	PORT2_PODR	20
4.5.2.24	PORTA_PDR	20
4.5.2.25	PORTA_PODR	20
4.5.2.26	PRCR	20
4.5.2.27	PWPR	21
4.5.2.28	REG16	21
4.5.2.29	REG32	21
4.5.2.30	REG8	21
4.5.3	Enumeration Type Documentation	22
4.5.3.1	pwm_const_t	22
4.5.4	Function Documentation	22
4.5.4.1	_tx_thread_context_restore()	22
4.5.4.2	_tx_thread_context_save()	22
4.5.4.3	clock_init()	23
4.5.4.4	cmt0_isr()	23
4.5.4.5	dbg_pulse()	24
4.5.4.6	delay_loops()	24
4.5.4.7	gptw0_init_pwm()	24
4.5.4.8	gptw0_pin_and_module_setup()	25
4.5.4.9	gptw_diag_dump()	26
4.5.4.10	heartbeat_n()	26
4.5.4.11	inline_clock_init()	27
4.5.4.12	main()	27
4.5.4.13	run_forever()	28
4.6	main.c	29

Chapter 1

Directory Hierarchy

1.1 Directories

pwm_test	5
clock.c	7
linker.ld	13
main.c	14

Chapter 2

File Index

2.1 File List

Here is a list of all files with brief descriptions:

pwm_test/ clock.c	
RX72N clock initialisation: HOCO 16 MHz -> PLL 192 -> ICLK 96 / PCKB 48 . . .	7
pwm_test/ linker.ld	13
pwm_test/ main.c	
Minimal 20 kHz PWM output on GPTW0 for AD2 scope verification	14

Chapter 3

Directory Documentation

3.1 pwm_test Directory Reference

Files

- file [clock.c](#)
RX72N clock initialisation: HOCO 16 MHz -> PLL 192 -> ICLK 96 / PCKB 48.
- file [linker.ld](#)
- file [main.c](#)
Minimal 20 kHz PWM output on GPTW0 for AD2 scope verification.

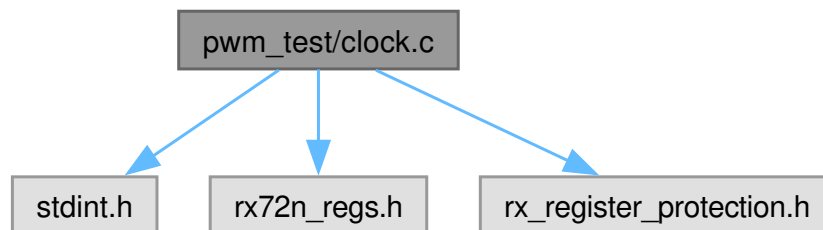
Chapter 4

File Documentation

4.1 pwm_test/clock.c File Reference

RX72N clock initialisation: HOCO 16 MHz -> PLL 192 -> ICLK 96 / PCKB 48.

```
#include <stdint.h>
#include "rx72n_regs.h"
#include "rx_register_protection.h"
Include dependency graph for clock.c:
```



Enumerations

- enum `clock_extra_addrs_t` : `uintptr_t` { `k_packcr_addr` = 0x00080044U }
- enum `clock_constants_word_t` : `uint16_t` { `k_pllcr_hoco_x12` = 0x1710U , `k_sckcr2_uck_div4` = 0x0031U , `k_packcr_pll_source` = 0x0000U }
- enum `clock_sckcr_t` : `uint32_t` { `k_sckcr_value` = 0x21C21211U }
- enum `clock_byte_constants_t` : `uint8_t` {
 `k_hococr_running` = 0x00U , `k_pllcr2_enable` = 0x00U , `k_memwait_one` = 0x01U ,
 `k_oscovfsr_hcovf` = 0x08U ,
 `k_oscovfsr_plovf` = 0x04U }

Functions

- static volatile `uint16_t` * `packcr_reg` (void)
- void `clock_init` (void)

4.1.1 Detailed Description

RX72N clock initialisation: HOCO 16 MHz -> PLL 192 -> ICLK 96 / PCKB 48.

This test sources the PLL from the internal HOCO (16 MHz) instead of the STAR PCB's 24 MHz external crystal on MOSC/EXTAL. The choice is deliberate (isolate the PLL bring-up from crystal startup), not because the board lacks a crystal – it has one.

Target clocks:

- PLL = HOCO 16 MHz x 12 = 192 MHz
- ICLK = 192 / 2 = 96 MHz (CPU)
- PCKA = 192 / 2 = 96 MHz
- PCKB = 192 / 4 = 48 MHz (CMT0 ThreadX tick)
- PCKC = 192 / 2 = 96 MHz
- PCKD = 192 / 2 = 96 MHz
- FCLK = 192 / 4 = 48 MHz (flash)
- BCLK = 192 / 4 = 48 MHz
- UCLK = 192 / 4 = 48 MHz (not used, but set for completeness)

Path: HOCO 16 MHz -> PLIDIV /1 -> STC x12 -> 192 MHz PLL output.

SPDX-License-Identifier: MIT

Copyright

Copyright (c) 2026 Locked Inc.

Definition in file [clock.c](#).

4.1.2 Enumeration Type Documentation

4.1.2.1 clock_byte_constants_t

enum [clock_byte_constants_t](#) : uint8_t

Enumerator

k_hococr_running	HOCOCCR: 0 = HOCO running
k_pllcr2_enable	PLLCCR2: 0 = PLL running (inverted logic)
k_memwait_one	MEMWAIT: 1 wait state for ICLK > 120 MHz
k_oscovfsr_hcovf	OSCOVFSR bit 3: HOCO stable
k_oscovfsr_plovf	OSCOVFSR bit 2: PLL stable

Definition at line 75 of file [clock.c](#).

```
00075      : uint8_t {
00076  k_hococr_running = 0x00U, /**< HOCOCCR: 0 = HOCO running */
00077  k_pllcr2_enable  = 0x00U, /**< PLLCCR2: 0 = PLL running (inverted logic) */
00078  k_memwait_one    = 0x01U, /**< MEMWAIT: 1 wait state for ICLK > 120 MHz */
00079  k_oscovfsr_hcovf = 0x08U, /**< OSCOVFSR bit 3: HOCO stable */
00080  k_oscovfsr_plovf = 0x04U, /**< OSCOVFSR bit 2: PLL stable */
00081 } clock_byte_constants_t;
```

4.1.2.2 clock_constants_word_t

enum [clock_constants_word_t](#) : uint16_t

Enumerator

k_pllcr_hoco_x12	
k_sckcr2_uck_div4	
k_packcr_pll_source	

Definition at line 43 of file [clock.c](#).

```

00043         : uint16_t {
00044     /*
00045      * PLLCR layout (16 bits):
00046      * bits [1:0] PLIDIV : 00=/1, 01=/2, 10=/3
00047      * bit [4] PLLSRCSEL: 0=MOSC, 1=HOCO
00048      * bits [13:8] STC : multiply = (STC + 1) / 2
00049      *
00050      * HOCO 16 MHz / 1 * 12 = 192 MHz PLL output.
00051      * STC = (12 * 2) - 1 = 23 = 0x17.
00052      * PLLSRCSEL = 1 (HOCO) -> bit 4 set.
00053      */
00054     k_pllcr_hoco_x12 = 0x1710U,
00055
00056     /*
00057      * SCKCR2: bits [7:4] UCK = /(N+1), bits [3:0] reserved (must be 0x1).
00058      * UCK = 0x3 -> /4 -> 192 MHz / 4 = 48 MHz exact.
00059      */
00060     k_sckcr2_uck_div4 = 0x0031U,
00061
00062     /* PACKCR: bit 0 UPLLSEL -- 0 routes UCLK divider input from main PLL. */
00063     k_packcr_pll_source = 0x0000U,
00064 } clock_constants_word_t;

```

4.1.2.3 clock_extra_addrs_t

enum [clock_extra_addrs_t](#) : uintptr_t

Enumerator

k_packcr_addr	Peripheral clock source: 0=PLL, 1=PPLL
---------------	--

Definition at line 36 of file [clock.c](#).

```

00036         : uintptr_t {
00037     k_packcr_addr = 0x00080044U, /**< Peripheral clock source: 0=PLL, 1=PPLL */
00038 } clock_extra_addrs_t;

```

4.1.2.4 clock_sckcr_t

enum [clock_sckcr_t](#) : uint32_t

Enumerator

k_sckcr_value	
---------------	--

Definition at line 66 of file [clock.c](#).

```

00066         : uint32_t {
00067     /*
00068      * SCKCR dividers:
00069      * FCK=/4 (48), ICK=/2 (96), PSTOP1=1, PSTOP0=1, BCK=/4 (48),
00070      * PCKA=/2 (96), PCKB=/4 (48), PCKC=/2 (96), PCKD=/2 (96).
00071      */
00072     k_sckcr_value = 0x21C21211U,
00073 } clock_sckcr_t;

```

4.1.3 Function Documentation

4.1.3.1 clock_init()

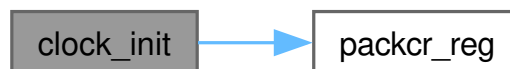
```
void clock_init (
    void )
```

Definition at line 100 of file [clock.c](#).

```
00101 {
00102     volatile rx_system_regs_t *sys = system_regs();
00103
00104     /* Step 0: unlock PRC0 (clock) + PRC1 (module stop) */
00105     *prcr_reg() = k_rx_prcr_unlock_prc0_prc1;
00106
00107     /* Step 1: start HOCO if not already running. */
00108     sys->hococr = k_hococr_running;
00109
00110     /* Step 2: wait for HOCO to stabilise. */
00111     while ((sys->oscofsr & k_oscofsr_hcovf) == 0U) {
00112         __asm__ volatile("nop");
00113     }
00114
00115     /* Step 3: program PLL: HOCO 16 MHz / 1 * 12 = 192 MHz, and enable. */
00116     sys->pllcr = k_pllcr_hoco_x12;
00117     sys->pllcr2 = k_pllcr2_enable;
00118
00119     /* Step 4: wait for PLL to stabilise. */
00120     while ((sys->oscofsr & k_oscofsr_plovf) == 0U) {
00121         __asm__ volatile("nop");
00122     }
00123
00124     /* Step 5: MANDATORY -- raise flash wait state before ICLK exceeds 120 MHz. */
00125     *memwait_reg() = k_memwait_one;
00126
00127     /* Step 6: program all bus dividers. */
00128     sys->skcr = k_skcr_value;
00129
00130     /* Step 7: select main PLL as the UCLK divider source (UPLLSEL=0). */
00131     *packcr_reg() = k_packcr_pll_source;
00132
00133     /* Step 8: program the USB clock divider. */
00134     sys->skcr2 = k_skcr2_uck_div4;
00135
00136     /* Step 9: switch the system clock source to PLL. */
00137     sys->skcr3 = k_skcr3_cksel_pll;
00138
00139     /* Step 10: relock PRCR. */
00140     *prcr_reg() = k_rx_prcr_lock;
00141 }
```

References [k_hococr_running](#), [k_memwait_one](#), [k_oscofsr_hcovf](#), [k_oscofsr_plovf](#), [k_packcr_pll_source](#), [k_pllcr2_enable](#), [k_pllcr_hoco_x12](#), [k_skcr2_uck_div4](#), [k_skcr_value](#), and [packcr_reg\(\)](#).

Here is the call graph for this function:



4.1.3.2 packcr_reg()

```
volatile uint16_t * packcr_reg (
    void ) [inline], [static]
```

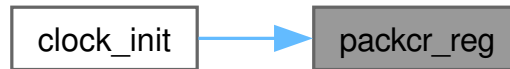
Definition at line 86 of file [clock.c](#).

```
00087 {
00088     return (volatile uint16_t *)k_packcr_addr;
00089 }
```

References [k_packcr_addr](#).

Referenced by [clock_init\(\)](#).

Here is the caller graph for this function:



4.2 clock.c

[Go to the documentation of this file.](#)

```

00001 /**
00002  * @file clock.c
00003  * @brief RX72N clock initialisation: HOCO 16 MHz -> PLL 192 -> ICLK 96 / PCKB 48.
00004  *
00005  * @details
00006  * This test sources the PLL from the internal HOCO (16 MHz) instead of
00007  * the STAR PCB's 24 MHz external crystal on MOSC/EXTAL. The choice is
00008  * deliberate (isolate the PLL bring-up from crystal startup), not because
00009  * the board lacks a crystal -- it has one.
00010  *
00011  * Target clocks:
00012  * - PLL = HOCO 16 MHz x 12 = 192 MHz
00013  * - ICLK = 192 / 2 = 96 MHz (CPU)
00014  * - PCKA = 192 / 2 = 96 MHz
00015  * - PCKB = 192 / 4 = 48 MHz (CMT0 ThreadX tick)
00016  * - PCKC = 192 / 2 = 96 MHz
00017  * - PCKD = 192 / 2 = 96 MHz
00018  * - FCLK = 192 / 4 = 48 MHz (flash)
00019  * - BCLK = 192 / 4 = 48 MHz
00020  * - UCLK = 192 / 4 = 48 MHz (not used, but set for completeness)
00021  *
00022  * Path: HOCO 16 MHz -> PLIDIV /1 -> STC x12 -> 192 MHz PLL output.
00023  *
00024  * SPDX-License-Identifier: MIT
00025  * @copyright Copyright (c) 2026 Locked Inc.
00026  */
00027
00028 #include <stdint.h>
00029
00030 #include "rx72n_regs.h"
00031 #include "rx_register_protection.h"
00032
00033 /*
00034  * Register addresses not exposed by the rx_system_regs_t struct
00035  */
00036
00037 typedef enum : uintptr_t {
00038     k_packcr_addr = 0x00080044U, /**< Peripheral clock source: 0=PLL, 1=PPLL */
00039 } clock_extra_addrs_t;
00040
00041 /*
00042  * Clock configuration constants
00043  */
00044
00045 typedef enum : uint16_t {
00046     /**
00047      * PLLCR layout (16 bits):
00048      * bits [1:0] PLIDIV : 00=/1, 01=/2, 10=/3
00049      * bit [4] PLLSRCSEL: 0=MOSC, 1=HOCO
00050      * bits [13:8] STC : multiply = (STC + 1) / 2
00051      *
00052      * HOCO 16 MHz / 1 * 12 = 192 MHz PLL output.
00053      * STC = (12 * 2) - 1 = 23 = 0x17.
00054      * PLLSRCSEL = 1 (HOCO) -> bit 4 set.
00055      */
00056     k_pllcr_hoco_x12 = 0x1710U,
00057
00058     /**
00059      * SCKCR2: bits [7:4] UCK = /(N+1), bits [3:0] reserved (must be 0x1).
00060      * UCK = 0x3 -> /4 -> 192 MHz / 4 = 48 MHz exact.
00061      */
00062     k_sckcr2_uck_div4 = 0x0031U,
00063
00064     /** PACKCR: bit 0 UPLLSEL -- 0 routes UCLK divider input from main PLL. */
00065     k_packcr_pll_source = 0x0000U,
00066 } clock_constants_word_t;
00067
00068 typedef enum : uint32_t {
00069     /**

```

```

00068  * SCKCR dividers:
00069  *   FCK=/4 (48), ICK=/2 (96), PSTOP1=1, PSTOP0=1, BCK=/4 (48),
00070  *   PCKA=/2 (96), PCKB=/4 (48), PCKC=/2 (96), PCKD=/2 (96).
00071  */
00072  k_sckcr_value = 0x21C21211U,
00073 } clock_sckcr_t;
00074
00075 typedef enum : uint8_t {
00076     k_hococr_running    = 0x00U, /**< HOCOCR: 0 = HOCO running */
00077     k_pllcr2_enable     = 0x00U, /**< PLLCR2: 0 = PLL running (inverted logic) */
00078     k_memwait_one       = 0x01U, /**< MEMWAIT: 1 wait state for ICLK > 120 MHz */
00079     k_oscovfsr_hcovf    = 0x08U, /**< OSCOVFSR bit 3: HOCO stable */
00080     k_oscovfsr_plovf    = 0x04U, /**< OSCOVFSR bit 2: PLL stable */
00081 } clock_byte_constants_t;
00082
00083 /*
=====
00084  * Inline accessors
00085  *
=====
*/
00086 static inline volatile uint16_t *packcr_reg(void)
00087 {
00088     return (volatile uint16_t *)k_packcr_addr;
00089 }
00090
00091 /*
=====
00092  * clock_init -- bring the chip from LOCO 240 kHz to PLL 192/96/48 MHz.
00093  *
00094  * Uses HOCO (16 MHz internal oscillator) as the PLL source.
00095  * No external crystal required.
00096  *
00097  * Must run before any peripheral that depends on PCKB is touched.
00098  * Called from main() before cmt0_init() and before tx_kernel_enter().
00099  *
=====
*/
00100 void clock_init(void)
00101 {
00102     volatile rx_system_regs_t *sys = system_regs();
00103
00104     /* Step 0: unlock PRC0 (clock) + PRC1 (module stop) */
00105     *prcr_reg() = k_rx_prcr_unlock_prc0_prc1;
00106
00107     /* Step 1: start HOCO if not already running. */
00108     sys->hococr = k_hococr_running;
00109
00110     /* Step 2: wait for HOCO to stabilise. */
00111     while ((sys->oscovfsr & k_oscovfsr_hcovf) == 0U) {
00112         __asm__ volatile("nop");
00113     }
00114
00115     /* Step 3: program PLL: HOCO 16 MHz / 1 * 12 = 192 MHz, and enable. */
00116     sys->pllcr = k_pllcr_hoco_x12;
00117     sys->pllcr2 = k_pllcr2_enable;
00118
00119     /* Step 4: wait for PLL to stabilise. */
00120     while ((sys->oscovfsr & k_oscovfsr_plovf) == 0U) {
00121         __asm__ volatile("nop");
00122     }
00123
00124     /* Step 5: MANDATORY -- raise flash wait state before ICLK exceeds 120 MHz. */
00125     *memwait_reg() = k_memwait_one;
00126
00127     /* Step 6: program all bus dividers. */
00128     sys->sckcr = k_sckcr_value;
00129
00130     /* Step 7: select main PLL as the UCLK divider source (UPLLSEL=0). */
00131     *packcr_reg() = k_packcr_pll_source;
00132
00133     /* Step 8: program the USB clock divider. */
00134     sys->sckcr2 = k_sckcr2_uck_div4;
00135
00136     /* Step 9: switch the system clock source to PLL. */
00137     sys->sckcr3 = k_sckcr3_cksel_pll;
00138
00139     /* Step 10: relock PRCR. */
00140     *prcr_reg() = k_rx_prcr_lock;
00141 }

```


4.3 pwm_test/linker.ld File Reference

4.4 linker.ld

[Go to the documentation of this file.](#)

```

00001 /*
00002  * gpio_test/linker.ld -- Linker script for GPIO breakout board test.
00003  *
00004  * Same memory map as usb_test: 512 KB internal SRAM, 2 MB internal ROM.
00005  * Includes Renesas-syntax sections (P, B, D, C) needed by ThreadX RXv3 port.
00006  */
00007
00008 MEMORY
00009 {
00010     RAM : ORIGIN = 0x00000004, LENGTH = 0x7FFFC /* 512 KB - 4 bytes */
00011     ROM : ORIGIN = 0xFFE00000, LENGTH = 0x200000 /* 2 MB */
00012 }
00013
00014 SECTIONS
00015 {
00016     /* Code -- explicit ROM base so section VMA is anchored correctly. */
00017     .text 0xFFE00000 : AT(0xFFE00000)
00018     {
00019         *(.text*)
00020         *(P)
00021         *(P_1)
00022         *(.rodata*)
00023         *(C)
00024         *(C_1)
00025         *(C_2)
00026         etext = .;
00027     } > ROM
00028
00029     /*
00030     * Relocatable vector table -- startup.S loads INTB with this address.
00031     * Must be 4-byte aligned; only vectors 27 (SWINT) and 28 (CMT0) are
00032     * used, but allocate 256 entries (1024 bytes) for safety.
00033     */
00034     .rvectors ALIGN(4) :
00035     {
00036         _rvectors_start = .;
00037         KEEP(*(.rvectors))
00038         _rvectors_end = .;
00039     } > ROM
00040
00041     /*
00042     * Initialised data -- LMA stays in ROM, VMA in RAM.
00043     * startup.S copies __data..edata from __mdata on boot.
00044     */
00045     __mdata = .;
00046     .data : AT(__mdata)
00047     {
00048         __data = .;
00049         *(.data*)
00050         *(D)
00051         *(D_1)
00052         *(D_2)
00053         __edata = .;
00054     } > RAM
00055
00056     /* BSS -- zeroed by startup.S */
00057     .bss :
00058     {
00059         __bss = .;
00060         *(.bss*)
00061         *(COMMON)
00062         *(B)
00063         *(B_1)
00064         *(B_2)
00065         __ebss = .;
00066         . = ALIGN(4);
00067         __end = .;
00068     } > RAM AT>RAM
00069
00070     /*
00071     * Stacks -- USP grows down from top of RAM, ISP 4 KB below that.
00072     */
00073     __ustack = ORIGIN(RAM) + LENGTH(RAM);
00074     __istack = __ustack - 0x1000;
00075
00076     /*
00077     * Exception vector table at fixed hardware address 0xFFFFF80.

```

```

00078      * 32 entries -- all unused.
00079      */
00080      .exvectors 0xFFFFF80 : AT(0xFFFFF80)
00081      {
00082          LONG(0xFFFFFFFF); LONG(0xFFFFFFFF); LONG(0xFFFFFFFF); LONG(0xFFFFFFFF);
00083          LONG(0xFFFFFFFF); LONG(0xFFFFFFFF); LONG(0xFFFFFFFF); LONG(0xFFFFFFFF);
00084          LONG(0xFFFFFFFF); LONG(0xFFFFFFFF); LONG(0xFFFFFFFF); LONG(0xFFFFFFFF);
00085          LONG(0xFFFFFFFF); LONG(0xFFFFFFFF); LONG(0xFFFFFFFF); LONG(0xFFFFFFFF);
00086          LONG(0xFFFFFFFF); LONG(0xFFFFFFFF); LONG(0xFFFFFFFF); LONG(0xFFFFFFFF);
00087          LONG(0xFFFFFFFF); LONG(0xFFFFFFFF); LONG(0xFFFFFFFF); LONG(0xFFFFFFFF);
00088          LONG(0xFFFFFFFF); LONG(0xFFFFFFFF); LONG(0xFFFFFFFF); LONG(0xFFFFFFFF);
00089          LONG(0xFFFFFFFF); LONG(0xFFFFFFFF); LONG(0xFFFFFFFF);
00090      } > ROM
00091
00092      /* Fixed reset vector at 0xFFFFF0C */
00093      .fvectors 0xFFFFF0C : AT(0xFFFFF0C)
00094      {
00095          KEEP*(.fvectors)
00096      } > ROM
00097 }

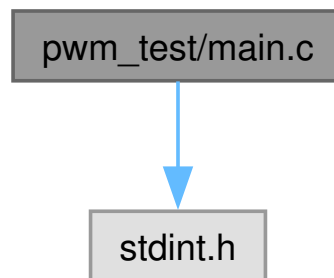
```

4.5 pwm_test/main.c File Reference

Minimal 20 kHz PWM output on GPTW0 for AD2 scope verification.

```
#include <stdint.h>
```

Include dependency graph for main.c:



Macros

- `#define REG8(a)`
- `#define REG16(a)`
- `#define REG32(a)`
- `#define PRCR 0x000803FEU`
- `#define MSTPCRA 0x00080010U /* bit 7 = GPTW (per Renesas iodef.h -- MST-PCRC.MSTPC6 is actually ECCRAM) */`
- `#define PWPR 0x0008C11FU`
- `#define P17PFS 0x0008C14FU /* pin 38 on Tom board */`
- `#define P23PFS 0x0008C153U /* pin 34 on Tom board */`
- `#define PORT1_PMR 0x0008C061U`
- `#define PORT2_PMR 0x0008C062U`
- `#define PORT1_PDR 0x0008C001U`
- `#define PORT2_PDR 0x0008C002U`
- `#define PORT1_PODR 0x0008C021U`
- `#define PORT2_PODR 0x0008C022U`
- `#define GPTW0_BASE 0x000C2000U`
- `#define GTWP (GPTW0_BASE + 0x00U) /* 32b: write protection */`
- `#define GTCR (GPTW0_BASE + 0x2CU) /* 32b: MD, TPCS, CST */`

- `#define GTUDDTYC (GPTW0_BASE + 0x30U) /* 32b: count direction */`
- `#define GTIOR (GPTW0_BASE + 0x34U) /* 32b: GTIOCA/B output config */`
- `#define GTINTAD (GPTW0_BASE + 0x38U)`
- `#define GTST (GPTW0_BASE + 0x3CU)`
- `#define GTBER (GPTW0_BASE + 0x40U) /* buffer enable */`
- `#define GTCNT (GPTW0_BASE + 0x48U) /* counter */`
- `#define GTCCRA (GPTW0_BASE + 0x4CU) /* duty A */`
- `#define GTCCRB (GPTW0_BASE + 0x50U) /* duty B */`
- `#define GTPR (GPTW0_BASE + 0x64U) /* period */`
- `#define GTPBR (GPTW0_BASE + 0x68U) /* period buffer */`
- `#define PORTA_PDR 0x0008C00AU`
- `#define PORTA_PODR 0x0008C02AU`
- `#define PA7_MASK 0x80U`

Enumerations

- `enum pwm_const_t : uint32_t {`
`k_pwm_pcka_hz = 96000000U , k_pwm_freq_hz = 20000U , k_pwm_period_cnt = 4800U ,`
`k_pwm_gtpr = 4799U ,`
`k_duty_step = 48U , k_duty_hold_loops = 40000U , k_psel_gtiocxa = 0x06U , k_psel_gtiocxb`
`= 0x06U }`

Functions

- `void clock_init (void)`
- `static void delay_loops (uint32_t n)`
- `static void gptw0_pin_and_module_setup (void)`
- `static void gptw0_init_pwm (void)`
- `static void gptw_diag_dump (void)`
- `static void run_forever (void)`
- `static void dbg_pulse (uint32_t n)`
- `static void inline_clock_init (void)`
- `static void heartbeat_n (uint32_t n)`
- `int main (void)`
- `void __tx_thread_context_save (void)`
- `void __tx_thread_context_restore (void)`
- `void cmt0_isr (void)`

4.5.1 Detailed Description

Minimal 20 kHz PWM output on GPTW0 for AD2 scope verification.

No motor, no H-bridge – just drives the Motor 0 PWM pins with a sawtooth-PWM duty sweep so the AD2 scope sees a clean 0-100% triangle envelope on the logic-level outputs.

Tom's PCB breakout mapping (see [gpio_test/pin_map.md](#)):

- Tom header pin 34 -> P23 / GTIOC0A -> "IN2" (scope Ch2, blue)
- Tom header pin 38 -> P17 / GTIOC0B -> "IN1" (scope Ch1, orange)

Clocks (from [clock.c](#)):

- $PLL = HOCO\ 16\ MHz * 12 = 192\ MHz$
- $PCKA = 192 / 2 = 96\ MHz$ (GPTW source)

PWM:

- Saw-wave mode (GTCR.MD=000)
- $Period = 96\ MHz / 20\ kHz = 4800\ counts \rightarrow GTPR = 4799$
- $GTCCRA = duty\ (0 \dots GTPR)$ for GTIOCA (P23)
- $GTCCRB = GTPR - duty$ for GTIOCB (P17) – complementary

Observable on the scope:

- 50 us period (20 kHz) on both channels
- 3.3 V logic levels (digital, not motor-driven)
- Duty cycle sweeps 0 -> 100% -> 0% every ~4 seconds
- Ch1 (P17) and Ch2 (P23) are mirror images

SPDX-License-Identifier: MIT

Copyright

Copyright (c) 2026 Locked Inc.

Definition in file [main.c](#).

4.5.2 Macro Definition Documentation

4.5.2.1 GPTW0_BASE

```
#define GPTW0_BASE 0x000C2000U
```

Definition at line 64 of file [main.c](#).

4.5.2.2 GTBER

```
#define GTBER (GPTW0_BASE + 0x40U) /* buffer enable */
```

Definition at line 71 of file [main.c](#).

Referenced by [gptw0_init_pwm\(\)](#).

4.5.2.3 GTCCRA

```
#define GTCCRA (GPTW0_BASE + 0x4CU) /* duty A */
```

Definition at line 73 of file [main.c](#).

Referenced by [gptw0_init_pwm\(\)](#), [main\(\)](#), and [run_forever\(\)](#).

4.5.2.4 GTCCRB

```
#define GTCCRB (GPTW0_BASE + 0x50U) /* duty B */
```

Definition at line 74 of file [main.c](#).

Referenced by [gptw0_init_pwm\(\)](#), [main\(\)](#), and [run_forever\(\)](#).

4.5.2.5 GTCNT

```
#define GTCNT (GPTW0_BASE + 0x48U) /* counter */
```

Definition at line 72 of file [main.c](#).

Referenced by [gptw0_init_pwm\(\)](#), [gptw_diag_dump\(\)](#), and [main\(\)](#).

4.5.2.6 GTCR

```
#define GTCR (GPTW0_BASE + 0x2CU) /* 32b: MD, TPCS, CST */
```

Definition at line 66 of file [main.c](#).

Referenced by [gptw0_init_pwm\(\)](#), and [gptw_diag_dump\(\)](#).

4.5.2.7 GTINTAD

```
#define GTINTAD (GPTW0_BASE + 0x38U)
```

Definition at line 69 of file [main.c](#).

4.5.2.8 GTIOR

```
#define GTIOR (GPTW0_BASE + 0x34U) /* 32b: GTIOCA/B output config */
```

Definition at line 68 of file [main.c](#).

Referenced by [gptw0_init_pwm\(\)](#).

4.5.2.9 GTPBR

```
#define GTPBR (GPTW0_BASE + 0x68U) /* period buffer */
```

Definition at line 76 of file [main.c](#).

Referenced by [gptw0_init_pwm\(\)](#).

4.5.2.10 GTPR

```
#define GTPR (GPTW0_BASE + 0x64U) /* period */
```

Definition at line 75 of file [main.c](#).

Referenced by [gptw0_init_pwm\(\)](#).

4.5.2.11 GTST

```
#define GTST (GPTW0_BASE + 0x3CU)
```

Definition at line 70 of file [main.c](#).

4.5.2.12 GTUDDTYC

```
#define GTUDDTYC (GPTW0_BASE + 0x30U) /* 32b: count direction */
```

Definition at line 67 of file [main.c](#).

Referenced by [gptw0_init_pwm\(\)](#).

4.5.2.13 GTWP

```
#define GTWP (GPTW0_BASE + 0x00U) /* 32b: write protection */
```

Definition at line 65 of file [main.c](#).

Referenced by [gptw0_init_pwm\(\)](#).

4.5.2.14 MSTPCRA

```
#define MSTPCRA 0x00080010U /* bit 7 = GPTW (per Renesas iodef.h -- MSTPCRC.MSTPC6 is actually ECCRAM) */
```

Definition at line 48 of file [main.c](#).

Referenced by [gptw0_pin_and_module_setup\(\)](#).

4.5.2.15 P17PFS

```
#define P17PFS 0x0008C14FU /* pin 38 on Tom board */
```

Definition at line 52 of file [main.c](#).

Referenced by [gptw0_pin_and_module_setup\(\)](#).

4.5.2.16 P23PFS

```
#define P23PFS 0x0008C153U /* pin 34 on Tom board */
```

Definition at line 53 of file [main.c](#).

Referenced by [gptw0_pin_and_module_setup\(\)](#).

4.5.2.17 PA7_MASK

```
#define PA7_MASK 0x80U
```

Definition at line 81 of file [main.c](#).

Referenced by [dbg_pulse\(\)](#), [heartbeat_n\(\)](#), [main\(\)](#), and [run_forever\(\)](#).

4.5.2.18 PORT1_PDR

```
#define PORT1_PDR 0x0008C001U
```

Definition at line 58 of file [main.c](#).

Referenced by [gptw_diag_dump\(\)](#).

4.5.2.19 PORT1_PMR

```
#define PORT1_PMR 0x0008C061U
```

Definition at line 56 of file [main.c](#).

Referenced by [gptw0_pin_and_module_setup\(\)](#), and [gptw_diag_dump\(\)](#).

4.5.2.20 PORT1_PODR

```
#define PORT1_PODR 0x0008C021U
```

Definition at line 60 of file [main.c](#).

Referenced by [gptw_diag_dump\(\)](#).

4.5.2.21 PORT2_PDR

```
#define PORT2_PDR 0x0008C002U
```

Definition at line 59 of file [main.c](#).

Referenced by [gptw_diag_dump\(\)](#).

4.5.2.22 PORT2_PMR

```
#define PORT2_PMR 0x0008C062U
```

Definition at line 57 of file [main.c](#).

Referenced by [gptw0_pin_and_module_setup\(\)](#), and [gptw_diag_dump\(\)](#).

4.5.2.23 PORT2_PODR

```
#define PORT2_PODR 0x0008C022U
```

Definition at line 61 of file [main.c](#).

Referenced by [gptw_diag_dump\(\)](#).

4.5.2.24 PORTA_PDR

```
#define PORTA_PDR 0x0008C00AU
```

Definition at line 79 of file [main.c](#).

Referenced by [dbg_pulse\(\)](#), [heartbeat_n\(\)](#), [main\(\)](#), and [run_forever\(\)](#).

4.5.2.25 PORTA_PODR

```
#define PORTA_PODR 0x0008C02AU
```

Definition at line 80 of file [main.c](#).

Referenced by [dbg_pulse\(\)](#), [heartbeat_n\(\)](#), [main\(\)](#), and [run_forever\(\)](#).

4.5.2.26 PRCR

```
#define PRCR 0x000803FEU
```

Definition at line 47 of file [main.c](#).

Referenced by [gptw0_pin_and_module_setup\(\)](#).

4.5.2.27 PWPR

```
#define PWPR 0x0008C11FU
```

Definition at line 51 of file [main.c](#).

Referenced by [gptw0_pin_and_module_setup\(\)](#).

4.5.2.28 REG16

```
#define REG16(  
    a)
```

Value:

```
*(volatile uint16_t *) (a)
```

Definition at line 39 of file [main.c](#).

Referenced by [gptw0_pin_and_module_setup\(\)](#), and [inline_clock_init\(\)](#).

4.5.2.29 REG32

```
#define REG32(  
    a)
```

Value:

```
*(volatile uint32_t *) (a)
```

Definition at line 40 of file [main.c](#).

Referenced by [gptw0_init_pwm\(\)](#), [gptw0_pin_and_module_setup\(\)](#), [gptw_diag_dump\(\)](#), [inline_clock_init\(\)](#), [main\(\)](#), and [run_forever\(\)](#).

4.5.2.30 REG8

```
#define REG8(  
    a)
```

Value:

```
*(volatile uint8_t *) (a)
```

Definition at line 38 of file [main.c](#).

Referenced by [dbg_pulse\(\)](#), [gptw0_pin_and_module_setup\(\)](#), [gptw_diag_dump\(\)](#), [heartbeat_n\(\)](#), [inline_clock_init\(\)](#), [main\(\)](#), and [run_forever\(\)](#).

4.5.3 Enumeration Type Documentation

4.5.3.1 pwm_const_t

enum [pwm_const_t](#) : uint32_t

Enumerator

k_pwm_pcka_hz	
k_pwm_freq_hz	
k_pwm_period_cnt	
k_pwm_gtpr	
k_duty_step	
k_duty_hold_loops	
k_psel_gtiocxa	
k_psel_gtiocxb	

Definition at line 86 of file [main.c](#).

```

00086      : uint32_t {
00087      k_pwm_pcka_hz    = 96000000U,
00088      k_pwm_freq_hz   = 20000U,
00089      k_pwm_period_cnt = 4800U,          /* 96 MHz / 20 kHz */
00090      k_pwm_gtpr       = 4799U,         /* period - 1 */
00091      k_duty_step      = 48U,           /* ~1% of period */
00092      k_duty_hold_loops = 40000U,       /* delay between duty steps */
00093      k_psel_gtiocxa   = 0x06U,        /* GTIOC0A, GTIOC0B alt function */
00094      k_psel_gtiocxb   = 0x06U,
00095 } pwm_const_t;

```

4.5.4 Function Documentation

4.5.4.1 __tx_thread_context_restore()

```
void __tx_thread_context_restore (
    void )
```

Definition at line 357 of file [main.c](#).

```
00357 {}
```

4.5.4.2 __tx_thread_context_save()

```
void __tx_thread_context_save (
    void )
```

Definition at line 356 of file [main.c](#).

```
00356 {}
```

4.5.4.3 clock_init()

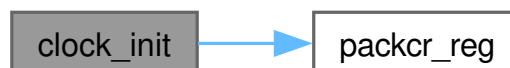
```
void clock_init (
    void ) [extern]
```

Definition at line 100 of file [clock.c](#).

```
00101 {
00102     volatile rx_system_regs_t *sys = system_regs();
00103
00104     /* Step 0: unlock PRC0 (clock) + PRC1 (module stop) */
00105     *prcr_reg() = k_rx_prcr_unlock_prc0_prc1;
00106
00107     /* Step 1: start HOCO if not already running. */
00108     sys->hococr = k_hococr_running;
00109
00110     /* Step 2: wait for HOCO to stabilise. */
00111     while ((sys->oscovfsr & k_oscovfsr_hcovf) == 0U) {
00112         __asm__ volatile("nop");
00113     }
00114
00115     /* Step 3: program PLL: HOCO 16 MHz / 1 * 12 = 192 MHz, and enable. */
00116     sys->pllcr = k_pllcr_hoco_x12;
00117     sys->pllcr2 = k_pllcr2_enable;
00118
00119     /* Step 4: wait for PLL to stabilise. */
00120     while ((sys->oscovfsr & k_oscovfsr_plovf) == 0U) {
00121         __asm__ volatile("nop");
00122     }
00123
00124     /* Step 5: MANDATORY -- raise flash wait state before ICLK exceeds 120 MHz. */
00125     *memwait_reg() = k_memwait_one;
00126
00127     /* Step 6: program all bus dividers. */
00128     sys->sckcr = k_sckcr_value;
00129
00130     /* Step 7: select main PLL as the UCLK divider source (UPLLSEL=0). */
00131     *packcr_reg() = k_packcr_pll_source;
00132
00133     /* Step 8: program the USB clock divider. */
00134     sys->sckcr2 = k_sckcr2_uck_div4;
00135
00136     /* Step 9: switch the system clock source to PLL. */
00137     sys->sckcr3 = k_sckcr3_cksel_pll;
00138
00139     /* Step 10: relock PRCR. */
00140     *prcr_reg() = k_rx_prcr_lock;
00141 }
```

References [k_hococr_running](#), [k_memwait_one](#), [k_oscovfsr_hcovf](#), [k_oscovfsr_plovf](#), [k_packcr_pll_source](#), [k_pllcr2_enable](#), [k_pllcr_hoco_x12](#), [k_sckcr2_uck_div4](#), [k_sckcr_value](#), and [packcr_reg\(\)](#).

Here is the call graph for this function:



4.5.4.4 cmt0_isr()

```
void cmt0_isr (
    void )
```

Definition at line 358 of file [main.c](#).

```
00358 {}
```

4.5.4.5 dbg_pulse()

```
void dbg_pulse (
    uint32_t n)  [static]
```

Definition at line 265 of file [main.c](#).

```
00266 {
00267     REG8(PORTA_PDR) |= PA7_MASK;
00268     for (uint32_t i = 0; i < n; i++) {
00269         REG8(PORTA_PODR) |= PA7_MASK;
00270         for (volatile uint32_t d = 0; d < 20000U; d++) { __asm__ volatile("nop"); }
00271         REG8(PORTA_PODR) &= (uint8_t)~PA7_MASK;
00272         for (volatile uint32_t d = 0; d < 20000U; d++) { __asm__ volatile("nop"); }
00273     }
00274     for (volatile uint32_t d = 0; d < 100000U; d++) { __asm__ volatile("nop"); }
00275 }
```

References [PA7_MASK](#), [PORTA_PDR](#), [PORTA_PODR](#), and [REG8](#).

4.5.4.6 delay_loops()

```
void delay_loops (
    uint32_t n)  [inline], [static]
```

Definition at line 97 of file [main.c](#).

```
00098 {
00099     for (volatile uint32_t i = 0U; i < n; i++) {
00100         __asm__ volatile("nop");
00101     }
00102 }
```

Referenced by [run_forever\(\)](#).

Here is the caller graph for this function:



4.5.4.7 gptw0_init_pwm()

```
void gptw0_init_pwm (
    void )  [static]
```

Definition at line 138 of file [main.c](#).

```
00139 {
00140     /* Unlock GTWP write-protection (key 0xA500 in upper bits, WP bits = 0). */
00141     REG32(GTWP) = 0x0000A500U;
00142
00143     /* Stop counter before configuring. */
00144     REG32(GTCR) = 0x00000000U;
00145     REG32(GTUDDTYC) = 0x00000003U; /* UD=1 (count up) + UDF=1 (forced update) -- without UDF, UD write
is ignored */
00146     REG32(GTCNT) = 0x00000000U;
00147
00148     /* Period: 4800 cycles at 96 MHz = 20 kHz. GTPR = period - 1. */
00149     REG32(GTPR) = (uint32_t)k_pwm_gtptr;
00150     REG32(GTPBR) = (uint32_t)k_pwm_gtptr;
00151
00152     /* Start at 50% duty for initial visibility. */
00153     REG32(GTCCR) = (uint32_t)(k_pwm_period_cnt / 2U);
00154     REG32(GTCCRB) = (uint32_t)(k_pwm_period_cnt - (k_pwm_period_cnt / 2U));
00155
00156     /* GTIOR per rx72n_gptw_regs.h:
```

```

00157  * GTIOA[4:0] = 0x09 -> initial LOW, high at cycle end, low at CMA
00158  * OAE bit 8 = 1 -> output enable on GTIOCA
00159  * GTIOB[20:16] = 0x09 same pattern
00160  * OBE bit 24 = 1 -> output enable on GTIOCB
00161  * (Complementary waveform is produced by setting GTCCRB = period-duty.)
00162  */
00163  /* GTIOA/GTIOB = 0x09: initial LOW, HIGH at cycle end (overflow), LOW
00164  * at compare match -> active-high sawtooth PWM, confirmed by HAL
00165  * k_gptw_gtior_oa_init_low and RX72N hw manual Ch.26 Table 26.22. */
00166  const uint32_t gtior =
00167      (0x09U « 0) | (1U « 8) | /* GTIOCA */
00168      (0x09U « 16) | (1U « 24); /* GTIOCB */
00169  REG32(GTIOR) = gtior;
00170
00171  /* No buffering. GTCCRA/GTCCRB writes take effect immediately. */
00172  REG32(GTBER) = 0x00000000U;
00173
00174  /* Start counter: MD=0 saw-wave, TPCS=0 (PCLKA/1), CST=1. */
00175  REG32(GTCR) = (0U « 16) | (0U « 24) | (1U « 0);
00176 }

```

References [GTBER](#), [GTCCRA](#), [GTCCRB](#), [GTCNT](#), [GTCR](#), [GTIOR](#), [GTPBR](#), [GTPR](#), [GTUDDTYC](#), [GTWP](#), [k_pwm_gtpr](#), [k_pwm_period_cnt](#), and [REG32](#).

Referenced by [main\(\)](#).

Here is the caller graph for this function:



4.5.4.8 gptw0_pin_and_module_setup()

```

void gptw0_pin_and_module_setup (
    void ) [static]

```

Definition at line 107 of file [main.c](#).

```

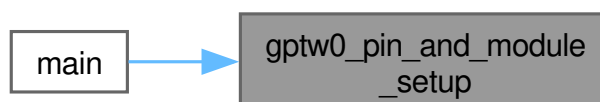
00108 {
00109  /* Unlock PRC1 (module stop), clear MSTPCRA.MSTPA7 (GPTW), lock. */
00110  REG16(PRCR) = 0xA502U;
00111  REG32(MSTPCRA) &= ~(1UL « 7);
00112  REG16(PRCR) = 0xA500U;
00113
00114  /* Enable PFS writes via MPC PWPR. */
00115  REG8(PWPR) = 0x00U; /* clear B0WI */
00116  REG8(PWPR) = 0x40U; /* set PFSWE */
00117
00118  /* P23 -> GTIOC0A, P17 -> GTIOC0B. PSEL = 0b011110 = 0x1E per
00119  * RX72N Hardware Manual Ch.23 Table 23.4 (P17) and Table 23.6 (P23).
00120  * The HAL's 0x14 is wrong for these specific pins -- 0x14 is reserved
00121  * on P17/P23 and leaves them Hi-Z. */
00122  REG8(P23PFS) = 0x1EU;
00123  REG8(P17PFS) = 0x1EU;
00124
00125  /* Re-protect PFS. */
00126  REG8(PWPR) = 0x00U;
00127  REG8(PWPR) = 0x80U;
00128
00129  /* Switch pins to peripheral mode (PMR=1). PDR stays input side
00130  * because the GTIOC output enable in GTIOR drives the pad. */
00131  REG8(PORT2_PMR) |= (uint8_t)(1U « 3); /* P23 */
00132  REG8(PORT1_PMR) |= (uint8_t)(1U « 7); /* P17 */
00133 }

```

References [MSTPCRA](#), [P17PFS](#), [P23PFS](#), [PORT1_PMR](#), [PORT2_PMR](#), [PRCR](#), [PWPR](#), [REG16](#), [REG32](#), and [REG8](#).

Referenced by [main\(\)](#).

Here is the caller graph for this function:



4.5.4.9 gptw_diag_dump()

```
void gptw_diag_dump (
    void ) [static]
```

Definition at line 182 of file [main.c](#).

```
00183 {
00184     volatile uint32_t cnt_a = REG32(GTCNT);
00185     for (volatile uint32_t d = 0; d < 1000U; d++) { __asm__ volatile("nop"); }
00186     volatile uint32_t cnt_b = REG32(GTCNT);
00187     volatile uint32_t cst = REG32(GTCR) & 1U;
00188     volatile uint32_t pmr1 = REG8(PORT1_PMR) & (1U << 7);
00189     volatile uint32_t pmr2 = REG8(PORT2_PMR) & (1U << 3);
00190
00191     /* Force P17/P23 back to GPIO mode so we can encode diagnostics on them. */
00192     REG8(PORT1_PMR) &= (uint8_t)~(1U << 7);
00193     REG8(PORT2_PMR) &= (uint8_t)~(1U << 3);
00194     REG8(PORT1_PDR) |= (1U << 7);
00195     REG8(PORT2_PDR) |= (1U << 3);
00196
00197     /* Encode diagnostics on Ch1 (P17) and Ch2 (P23):
00198     * - 5 pulses = GTCNT advanced (counter running)
00199     * - 3 pulses = GTCNT frozen
00200     * - 7 pulses = CST readback = 1
00201     * - 1 pulse = CST readback = 0
00202     * Sequence: cnt-state pulses, pause, cst pulses, pause, pmr pulses. */
00203     uint32_t counts_advanced = (cnt_b != cnt_a);
00204     uint32_t pulses_a = counts_advanced ? 5U : 3U;
00205     uint32_t pulses_b = cst ? 7U : 1U;
00206     uint32_t pulses_c = (pmr1 != 0U && pmr2 != 0U) ? 4U : 2U;
00207
00208     for (uint32_t round = 0; round < 3U; round++) {
00209         uint32_t n = (round == 0U) ? pulses_a : (round == 1U) ? pulses_b : pulses_c;
00210         for (uint32_t i = 0; i < n; i++) {
00211             REG8(PORT1_PODR) |= (1U << 7);
00212             REG8(PORT2_PODR) |= (1U << 3);
00213             for (volatile uint32_t d = 0; d < 30000U; d++) { __asm__ volatile("nop"); }
00214             REG8(PORT1_PODR) &= (uint8_t)~(1U << 7);
00215             REG8(PORT2_PODR) &= (uint8_t)~(1U << 3);
00216             for (volatile uint32_t d = 0; d < 30000U; d++) { __asm__ volatile("nop"); }
00217         }
00218         for (volatile uint32_t d = 0; d < 400000U; d++) { __asm__ volatile("nop"); }
00219     }
00220 }
```

References [GTCNT](#), [GTCR](#), [PORT1_PDR](#), [PORT1_PMR](#), [PORT1_PODR](#), [PORT2_PDR](#), [PORT2_PMR](#), [PORT2_PODR](#), [REG32](#), and [REG8](#).

4.5.4.10 heartbeat_n()

```
void heartbeat_n (
    uint32_t n) [static]
```

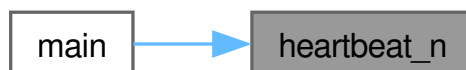
Definition at line 305 of file [main.c](#).

```
00306 {
00307     REG8(PORTA_PDR) |= PA7_MASK;
00308     for (uint32_t i = 0; i < n; i++) {
00309         REG8(PORTA_PODR) |= PA7_MASK;
00310         for (volatile uint32_t d = 0; d < 50000U; d++) { __asm__ volatile("nop"); }
00311         REG8(PORTA_PODR) &= (uint8_t)~PA7_MASK;
00312         for (volatile uint32_t d = 0; d < 50000U; d++) { __asm__ volatile("nop"); }
00313     }
00314     for (volatile uint32_t d = 0; d < 200000U; d++) { __asm__ volatile("nop"); }
00315 }
```

References [PA7_MASK](#), [PORTA_PDR](#), [PORTA_PODR](#), and [REG8](#).

Referenced by [main\(\)](#).

Here is the caller graph for this function:



4.5.4.11 inline_clock_init()

```
void inline_clock_init (
    void ) [static]
```

Definition at line 282 of file [main.c](#).

```
00283 {
00284     REG16(0x000803FEU) = 0xA503U;          /* PRC0+PRC1 unlock */
00285
00286     REG8(0x00080036U) = 0x00U;             /* HOCOCCR: start HOCO */
00287     while ((REG8(0x0008003CU) & 0x08U) == 0U) { __asm__ volatile("nop"); } /* HCOVF */
00288
00289     REG16(0x00080028U) = 0x1710U;          /* PLLCR: HOCO*12, PLIDIV=/1 */
00290     REG8(0x0008002AU) = 0x00U;            /* PLLCR2: enable */
00291     while ((REG8(0x0008003CU) & 0x04U) == 0U) { __asm__ volatile("nop"); } /* PLOVF */
00292
00293     REG8(0x0008101CU) = 0x01U;             /* MEMWAIT = 1 */
00294     REG32(0x00080020U) = 0x21C21211U;     /* SCKCR: ICLK/2 PCKA/2 PCKB/4 */
00295     REG16(0x00080044U) = 0x0001U;         /* PACKCR: bit 0 reserved must be 1 */
00296     REG16(0x00080024U) = 0x0031U;         /* SCKCR2: UCK /4 */
00297     REG16(0x00080026U) = 0x0400U;         /* SCKCR3: CKSEL=PLL */
00298
00299     REG16(0x000803FEU) = 0xA500U;          /* PRCR lock */
00300 }
```

References [REG16](#), [REG32](#), and [REG8](#).

Referenced by [main\(\)](#).

Here is the caller graph for this function:



4.5.4.12 main()

```
int main (
    void )
```

Definition at line 317 of file [main.c](#).

```
00318 {
00319     /* PA7 as diagnostic output; do not touch P17/P23 as GPIO after this. */
00320     REG8(PORTA_PDR) |= PA7_MASK;
00321     REG8(PORTA_PODR) &= (uint8_t)~PA7_MASK;
00322
00323     heartbeat_n(1);          /* 1: main() entered */
00324     inline_clock_init();
00325     heartbeat_n(2);          /* 2: clock_init returned */
00326
00327     gptw0_pin_and_module_setup(); /* sets PMR=1 on P17+P23 */
00328     heartbeat_n(3);          /* 3: pin/module setup done */
00329
00330     gptw0_init_pwm();        /* configure + start counter */
00331     heartbeat_n(4);          /* 4: GPTW init done */
00332
00333     volatile uint32_t a = REG32(GTCNT);
00334     for (volatile uint32_t d = 0; d < 10000U; d++) { __asm__ volatile("nop"); }
00335     volatile uint32_t b = REG32(GTCNT);
00336     heartbeat_n((b != a) ? 7U : 5U); /* 7: counter running, 5: frozen */
00337
00338     /* Lock in 50% duty for clean scope capture. No sweep. */
00339     REG32(GTCCRA) = (uint32_t)(k_pwm_period_cnt / 2U);
00340     REG32(GTCCRB) = (uint32_t)(k_pwm_period_cnt - (k_pwm_period_cnt / 2U));
00341
00342     /* PA7 heartbeat @ slow rate so we can see firmware is alive. */
00343     REG8(PORTA_PDR) |= PA7_MASK;
00344     for (;;) {
00345         REG8(PORTA_PODR) ^= PA7_MASK;
00346         for (volatile uint32_t d = 0; d < 500000U; d++) { __asm__ volatile("nop"); }

```

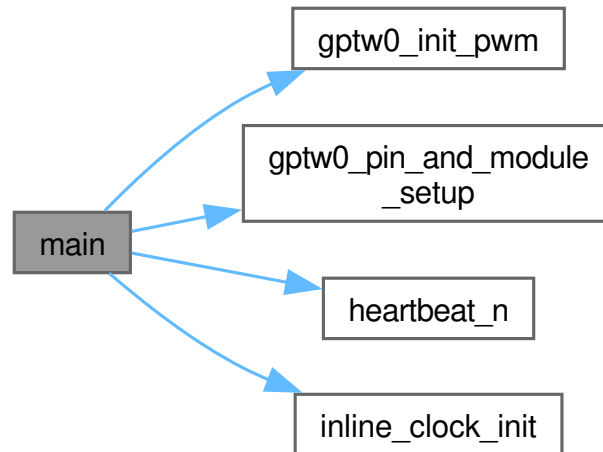
```

00347 }
00348 return 0;
00349 }

```

References [gptw0_init_pwm\(\)](#), [gptw0_pin_and_module_setup\(\)](#), [GTCCRA](#), [GTCCRB](#), [GTCNT](#), [heartbeat_n\(\)](#), [inline_clock_init\(\)](#), [k_pwm_period_cnt](#), [PA7_MASK](#), [PORTA_PDR](#), [PORTA_PODR](#), [REG32](#), and [REG8](#).

Here is the call graph for this function:



4.5.4.13 run_forever()

```

void run_forever (
    void ) [static]

```

Definition at line 226 of file [main.c](#).

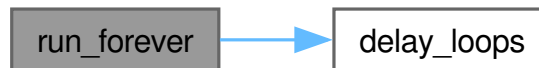
```

00227 {
00228     /* PA7 heartbeat LED. */
00229     REG8(PORTA_PDR) |= PA7_MASK;
00230     REG8(PORTA_PODR) &= (uint8_t)~PA7_MASK;
00231
00232     uint32_t duty = 0U;
00233     uint32_t direction = 1U; /* 1 = up, 0 = down */
00234
00235     for (;;) {
00236         /* Update duty on both channels. CCRB = period - CCRA keeps them
00237          * complementary (visible as mirror-image waveforms on the scope). */
00238         REG32(GTCCRA) = duty;
00239         REG32(GTCCRB) = (uint32_t)k_pwm_period_cnt - duty;
00240
00241         /* Toggle LED every loop so we can see the firmware is alive. */
00242         REG8(PORTA_PODR) ^= PA7_MASK;
00243
00244         delay_loops(k_duty_hold_loops);
00245
00246         if (direction != 0U) {
00247             duty += (uint32_t)k_duty_step;
00248             if (duty >= (uint32_t)k_pwm_period_cnt) {
00249                 duty = (uint32_t)k_pwm_period_cnt;
00250                 direction = 0U;
00251             }
00252         } else {
00253             if (duty <= (uint32_t)k_duty_step) {
00254                 duty = 0U;
00255                 direction = 1U;
00256             } else {
00257                 duty -= (uint32_t)k_duty_step;
00258             }
00259         }
00260     }
00261 }

```

References [delay_loops\(\)](#), [GTCCRA](#), [GTCCRB](#), [k_duty_hold_loops](#), [k_duty_step](#), [k_pwm_period_cnt](#), [PA7_MASK](#), [PORTA_PDR](#), [PORTA_PODR](#), [REG32](#), and [REG8](#).

Here is the call graph for this function:



4.6 main.c

[Go to the documentation of this file.](#)

```

00001 /**
00002  * @file main.c
00003  * @brief Minimal 20 kHz PWM output on GPTW0 for AD2 scope verification.
00004  *
00005  * @details
00006  * No motor, no H-bridge -- just drives the Motor 0 PWM pins with a
00007  * sawtooth-PWM duty sweep so the AD2 scope sees a clean 0-100% triangle
00008  * envelope on the logic-level outputs.
00009  *
00010  * Tom's PCB breakout mapping (see gpio_test/pin_map.md):
00011  * - Tom header pin 34 -> P23 / GTIOC0A -> "IN2" (scope Ch2, blue)
00012  * - Tom header pin 38 -> P17 / GTIOC0B -> "IN1" (scope Ch1, orange)
00013  *
00014  * Clocks (from clock.c):
00015  * - PLL = HOCO 16 MHz * 12 = 192 MHz
00016  * - PCKA = 192 / 2 = 96 MHz (GPTW source)
00017  *
00018  * PWM:
00019  * - Saw-wave mode (GTCR.MD=000)
00020  * - Period = 96 MHz / 20 kHz = 4800 counts -> GTPR = 4799
00021  * - GTCCRA = duty (0 .. GTPR) for GTIOCA (P23)
00022  * - GTCCRB = GTPR - duty for GTIOCB (P17) -- complementary
00023  *
00024  * Observable on the scope:
00025  * - 50 us period (20 kHz) on both channels
00026  * - 3.3 V logic levels (digital, not motor-driven)
00027  * - Duty cycle sweeps 0 -> 100% -> 0% every ~4 seconds
00028  * - Ch1 (P17) and Ch2 (P23) are mirror images
00029  *
00030  * SPDX-License-Identifier: MIT
00031  * @copyright Copyright (c) 2026 Locked Inc.
00032  */
00033
00034 #include <stdint.h>
00035
00036 extern void clock_init(void);
00037
00038 #define REG8(a) (*(volatile uint8_t *) (a))
00039 #define REG16(a) (*(volatile uint16_t *) (a))
00040 #define REG32(a) (*(volatile uint32_t *) (a))
00041
00042 /**
=====
00043  * Register addresses
00044  *
=====
00045  */
00046 /* System / module stop */
00047 #define PRCR 0x000803FEU
00048 #define MSTPCRA 0x00080010U /* bit 7 = GPTW (per Renesas iodef.h -- MSTPCRC.MSTPC6 is actually
ECCRAM) */
00049
00050 /* MPC pin function select */
00051 #define PWPR 0x0008C11FU
00052 #define P17PFS 0x0008C14FU /* pin 38 on Tom board */
00053 #define P23PFS 0x0008C153U /* pin 34 on Tom board */
00054
00055 /* PORT 1 (P17) and PORT 2 (P23) peripheral mode */
00056 #define PORT1_PMR 0x0008C061U
00057 #define PORT2_PMR 0x0008C062U
00058 #define PORT1_PDR 0x0008C001U
00059 #define PORT2_PDR 0x0008C002U
00060 #define PORT1_PODR 0x0008C021U
00061 #define PORT2_PODR 0x0008C022U
00062
00063 /* GPTW0 channel registers (base 0x000C2000) */
00064 #define GPTW0_BASE 0x000C2000U
00065 #define GTWP (GPTW0_BASE + 0x00U) /* 32b: write protection */
00066 #define GTCR (GPTW0_BASE + 0x2CU) /* 32b: MD, TPCS, CST */
00067 #define GTUDDTYC (GPTW0_BASE + 0x30U) /* 32b: count direction */
00068 #define GTIOR (GPTW0_BASE + 0x34U) /* 32b: GTIOCA/B output config */
00069 #define GTINTAD (GPTW0_BASE + 0x38U)

```

```

00070 #define GTST      (GPTW0_BASE + 0x3CU)
00071 #define GTBER      (GPTW0_BASE + 0x40U) /* buffer enable */
00072 #define GTCNT      (GPTW0_BASE + 0x48U) /* counter */
00073 #define GTCCRA      (GPTW0_BASE + 0x4CU) /* duty A */
00074 #define GTCCRB      (GPTW0_BASE + 0x50U) /* duty B */
00075 #define GTPR        (GPTW0_BASE + 0x64U) /* period */
00076 #define GTPBR        (GPTW0_BASE + 0x68U) /* period buffer */
00077
00078 /* Heartbeat LED on PA7 (pin 88) so we see firmware is alive. */
00079 #define PORTA_PDR 0x0008C00AU
00080 #define PORTA_PODR 0x0008C02AU
00081 #define PA7_MASK 0x80U
00082
00083 /*
=====
00084 * PWM configuration constants
00085 *
=====
*/
00086 typedef enum : uint32_t {
00087     k_pwm_pcka_hz = 96000000U,
00088     k_pwm_freq_hz = 20000U,
00089     k_pwm_period_cnt = 4800U, /* 96 MHz / 20 kHz */
00090     k_pwm_gtptr = 4799U, /* period - 1 */
00091     k_duty_step = 48U, /* ~1% of period */
00092     k_duty_hold_loops = 40000U, /* delay between duty steps */
00093     k_psel_gtiocxa = 0x06U, /* GTIOC0A, GTIOC0B alt function */
00094     k_psel_gtiocxb = 0x06U,
00095 } pwm_const_t;
00096
00097 static inline void delay_loops(uint32_t n)
00098 {
00099     for (volatile uint32_t i = 0U; i < n; i++) {
00100         __asm__ volatile("nop");
00101     }
00102 }
00103
00104 /*
=====
00105 * Bring GPTW0 out of module stop and wire P23/P17 to GTIOC0A/GTIOC0B.
00106 *
=====
*/
00107 static void gptw0_pin_and_module_setup(void)
00108 {
00109     /* Unlock PRC1 (module stop), clear MSTPCRA.MSTPA7 (GPTW), lock. */
00110     REG16(PRCR) = 0xA502U;
00111     REG32(MSTPCRA) &= ~(1UL << 7);
00112     REG16(PRCR) = 0xA500U;
00113
00114     /* Enable PFS writes via MPC PWPR. */
00115     REG8(PWPR) = 0x00U; /* clear B0WI */
00116     REG8(PWPR) = 0x40U; /* set PFSWE */
00117
00118     /* P23 -> GTIOC0A, P17 -> GTIOC0B. PSEL = 0b011110 = 0x1E per
00119      * RX72N Hardware Manual Ch.23 Table 23.4 (P17) and Table 23.6 (P23).
00120      * The HAL's 0x14 is wrong for these specific pins -- 0x14 is reserved
00121      * on P17/P23 and leaves them Hi-Z. */
00122     REG8(P23PFS) = 0x1EU;
00123     REG8(P17PFS) = 0x1EU;
00124
00125     /* Re-protect PFS. */
00126     REG8(PWPR) = 0x00U;
00127     REG8(PWPR) = 0x80U;
00128
00129     /* Switch pins to peripheral mode (PMR=1). PDR stays input side
00130      * because the GTIOC output enable in GTIOR drives the pad. */
00131     REG8(PORT2_PMR) |= (uint8_t)(1U << 3); /* P23 */
00132     REG8(PORT1_PMR) |= (uint8_t)(1U << 7); /* P17 */
00133 }
00134
00135 /*
=====
00136 * Configure GPTW0 as 20 kHz saw-wave PWM with complementary A/B outputs.
00137 *
=====
*/
00138 static void gptw0_init_pwm(void)
00139 {
00140     /* Unlock GTWP write-protection (key 0xA500 in upper bits, WP bits = 0). */
00141     REG32(GTWP) = 0x0000A500U;
00142
00143     /* Stop counter before configuring. */
00144     REG32(GTCR) = 0x00000000U;
00145     REG32(GTUDDTYC) = 0x00000003U; /* UD=1 (count up) + UDF=1 (forced update) -- without UDF, UD write
is ignored */
00146     REG32(GTCNT) = 0x00000000U;

```

```

00147
00148 /* Period: 4800 cycles at 96 MHz = 20 kHz. GTPR = period - 1. */
00149 REG32(GTPR) = (uint32_t)k_pwm_gtp;
00150 REG32(GTPBR) = (uint32_t)k_pwm_gtp;
00151
00152 /* Start at 50% duty for initial visibility. */
00153 REG32(GTCCRA) = (uint32_t)(k_pwm_period_cnt / 2U);
00154 REG32(GTCCRB) = (uint32_t)(k_pwm_period_cnt - (k_pwm_period_cnt / 2U));
00155
00156 /* GTIOR per rx72n_gptw_regs.h:
00157 * GTIOA[4:0] = 0x09 -> initial LOW, high at cycle end, low at CMA
00158 * OAE bit 8 = 1 -> output enable on GTIOCA
00159 * GTIOB[20:16] = 0x09 same pattern
00160 * OBE bit 24 = 1 -> output enable on GTIOCB
00161 * (Complementary waveform is produced by setting GTCCRB = period-duty.)
00162 */
00163 /* GTIOA/GTIOB = 0x09: initial LOW, HIGH at cycle end (overflow), LOW
00164 * at compare match -> active-high sawtooth PWM, confirmed by HAL
00165 * k_gptw_gtior_oa_init_low and RX72N hw manual Ch.26 Table 26.22. */
00166 const uint32_t gtior =
00167     (0x09U << 0) | (1U << 8) | /* GTIOCA */
00168     (0x09U << 16) | (1U << 24); /* GTIOCB */
00169 REG32(GTIOR) = gtior;
00170
00171 /* No buffering. GTCCRA/GTCCRB writes take effect immediately. */
00172 REG32(GTBER) = 0x00000000U;
00173
00174 /* Start counter: MD=0 saw-wave, TPCS=0 (PCLKA/1), CST=1. */
00175 REG32(GTCR) = (0U << 16) | (0U << 24) | (1U << 0);
00176 }
00177
00178 /* Debug helper: bring out two GPIOs (P17 and P23 as GPIO override) that
00179 * mirror whether GTCNT is nonzero and whether GTCR CST bit reads back.
00180 * WARNING: this re-purposes the PWM pins so scope will see GPIO-style
00181 * toggling not PWM. Use only for debug diagnosis. */
00182 static void gptw_diag_dump(void)
00183 {
00184     volatile uint32_t cnt_a = REG32(GTCNT);
00185     for (volatile uint32_t d = 0; d < 1000U; d++) { __asm__ volatile("nop"); }
00186     volatile uint32_t cnt_b = REG32(GTCNT);
00187     volatile uint32_t cst = REG32(GTCR) & 1U;
00188     volatile uint32_t pmr1 = REG8(PORT1_PMR) & (1U << 7);
00189     volatile uint32_t pmr2 = REG8(PORT2_PMR) & (1U << 3);
00190
00191     /* Force P17/P23 back to GPIO mode so we can encode diagnostics on them. */
00192     REG8(PORT1_PMR) &= (uint8_t)~(1U << 7);
00193     REG8(PORT2_PMR) &= (uint8_t)~(1U << 3);
00194     REG8(PORT1_PDR) |= (1U << 7);
00195     REG8(PORT2_PDR) |= (1U << 3);
00196
00197     /* Encode diagnostics on Ch1 (P17) and Ch2 (P23):
00198     * - 5 pulses = GTCNT advanced (counter running)
00199     * - 3 pulses = GTCNT frozen
00200     * - 7 pulses = CST readback = 1
00201     * - 1 pulse = CST readback = 0
00202     * Sequence: cnt-state pulses, pause, cst pulses, pause, pmr pulses. */
00203     uint32_t counts_advanced = (cnt_b != cnt_a);
00204     uint32_t pulses_a = counts_advanced ? 5U : 3U;
00205     uint32_t pulses_b = cst ? 7U : 1U;
00206     uint32_t pulses_c = (pmr1 != 0U && pmr2 != 0U) ? 4U : 2U;
00207
00208     for (uint32_t round = 0; round < 3U; round++) {
00209         uint32_t n = (round == 0U) ? pulses_a : (round == 1U) ? pulses_b : pulses_c;
00210         for (uint32_t i = 0; i < n; i++) {
00211             REG8(PORT1_PODR) |= (1U << 7);
00212             REG8(PORT2_PODR) |= (1U << 3);
00213             for (volatile uint32_t d = 0; d < 30000U; d++) { __asm__ volatile("nop"); }
00214             REG8(PORT1_PODR) &= (uint8_t)~(1U << 7);
00215             REG8(PORT2_PODR) &= (uint8_t)~(1U << 3);
00216             for (volatile uint32_t d = 0; d < 30000U; d++) { __asm__ volatile("nop"); }
00217         }
00218         for (volatile uint32_t d = 0; d < 40000U; d++) { __asm__ volatile("nop"); }
00219     }
00220 }
00221
00222 /*
=====
00223 * LED + PWM duty sweep forever. Produces a visible 0..100% triangle on the
00224 * scope so you can watch the duty cycle move.
00225 *
=====
*/
00226 static void run_forever(void)
00227 {
00228     /* PA7 heartbeat LED. */
00229     REG8(PORTA_PDR) |= PA7_MASK;
00230     REG8(PORTA_PODR) &= (uint8_t)~PA7_MASK;

```

```

00231
00232 uint32_t duty      = 0U;
00233 uint32_t direction = 1U; /* 1 = up, 0 = down */
00234
00235 for (;;) {
00236     /* Update duty on both channels. CCRB = period - CCRA keeps them
00237     * complementary (visible as mirror-image waveforms on the scope). */
00238     REG32(GTCCRA) = duty;
00239     REG32(GTCCRB) = (uint32_t)k_pwm_period_cnt - duty;
00240
00241     /* Toggle LED every loop so we can see the firmware is alive. */
00242     REG8(PORTA_PODR) ^= PA7_MASK;
00243
00244     delay_loops(k_duty_hold_loops);
00245
00246     if (direction != 0U) {
00247         duty += (uint32_t)k_duty_step;
00248         if (duty >= (uint32_t)k_pwm_period_cnt) {
00249             duty      = (uint32_t)k_pwm_period_cnt;
00250             direction = 0U;
00251         }
00252     } else {
00253         if (duty <= (uint32_t)k_duty_step) {
00254             duty      = 0U;
00255             direction = 1U;
00256         } else {
00257             duty -= (uint32_t)k_duty_step;
00258         }
00259     }
00260 }
00261 }
00262
00263 /* Slow toggle on PA7 LED so AD2 can see how far we got.
00264 * Probe a 3rd channel to PA7 / pin 88 and count pulses. */
00265 static void dbg_pulse(uint32_t n)
00266 {
00267     REG8(PORTA_PDR) |= PA7_MASK;
00268     for (uint32_t i = 0; i < n; i++) {
00269         REG8(PORTA_PODR) |= PA7_MASK;
00270         for (volatile uint32_t d = 0; d < 20000U; d++) { __asm__ volatile("nop"); }
00271         REG8(PORTA_PODR) &= (uint8_t)~PA7_MASK;
00272         for (volatile uint32_t d = 0; d < 20000U; d++) { __asm__ volatile("nop"); }
00273     }
00274     for (volatile uint32_t d = 0; d < 100000U; d++) { __asm__ volatile("nop"); }
00275 }
00276
00277 /*
=====
00278 * Clock setup verified working on Tom's PCB (no external crystal).
00279 * HOCO 16 MHz internal -> PLL x12 = 192 MHz, PCKA /2 = 96 MHz.
00280 * Copied from an earlier HOCO PID bring-up test that enumerated 1209:0002 on this
00281 * hardware, so these exact register writes are known-good. */
00282 static void inline_clock_init(void)
00283 {
00284     REG16(0x000803FEU) = 0xA503U; /* PRC0+PRC1 unlock */
00285
00286     REG8(0x00080036U) = 0x00U; /* HOCOCCR: start HOCO */
00287     while ((REG8(0x0008003CU) & 0x08U) == 0U) { __asm__ volatile("nop"); } /* HCOVF */
00288
00289     REG16(0x00080028U) = 0x1710U; /* PLLCR: HOCO*12, PLIDIV=/1 */
00290     REG8(0x0008002AU) = 0x00U; /* PLLCR2: enable */
00291     while ((REG8(0x0008003CU) & 0x04U) == 0U) { __asm__ volatile("nop"); } /* PLOVF */
00292
00293     REG8(0x0008101CU) = 0x01U; /* MEMWAIT = 1 */
00294     REG32(0x00080020U) = 0x21C21211U; /* SCKCR: ICLK/2 PCKA/2 PCKB/4 */
00295     REG16(0x00080044U) = 0x0001U; /* PACKCR: bit 0 reserved must be 1 */
00296     REG16(0x00080024U) = 0x0031U; /* SCKCR2: UCK /4 */
00297     REG16(0x00080026U) = 0x0400U; /* SCKCR3: CKSEL=PLL */
00298
00299     REG16(0x000803FEU) = 0xA500U; /* PRCR lock */
00300 }
00301
00302 /* Heartbeat/diagnostic pulses on PA7 only -- do NOT touch P17/P23 after
00303 * GPTW takes them in peripheral mode, or we clobber PMR and the PWM
00304 * output stops reaching the pad. */
00305 static void heartbeat_n(uint32_t n)
00306 {
00307     REG8(PORTA_PDR) |= PA7_MASK;
00308     for (uint32_t i = 0; i < n; i++) {
00309         REG8(PORTA_PODR) |= PA7_MASK;
00310         for (volatile uint32_t d = 0; d < 50000U; d++) { __asm__ volatile("nop"); }
00311         REG8(PORTA_PODR) &= (uint8_t)~PA7_MASK;
00312         for (volatile uint32_t d = 0; d < 50000U; d++) { __asm__ volatile("nop"); }
00313     }
00314     for (volatile uint32_t d = 0; d < 200000U; d++) { __asm__ volatile("nop"); }
00315 }
00316

```

```

00317 int main(void)
00318 {
00319     /* PA7 as diagnostic output; do not touch P17/P23 as GPIO after this. */
00320     REG8(PORTA_PDR) |= PA7_MASK;
00321     REG8(PORTA_PODR) &= (uint8_t)~PA7_MASK;
00322
00323     heartbeat_n(1);                /* 1: main() entered */
00324     inline_clock_init();
00325     heartbeat_n(2);                /* 2: clock_init returned */
00326
00327     gptw0_pin_and_module_setup(); /* sets PMR=1 on P17+P23 */
00328     heartbeat_n(3);                /* 3: pin/module setup done */
00329
00330     gptw0_init_pwm();              /* configure + start counter */
00331     heartbeat_n(4);                /* 4: GPTW init done */
00332
00333     volatile uint32_t a = REG32(GTCNT);
00334     for (volatile uint32_t d = 0; d < 10000U; d++) { __asm__ volatile("nop"); }
00335     volatile uint32_t b = REG32(GTCNT);
00336     heartbeat_n((b != a) ? 7U : 5U); /* 7: counter running, 5: frozen */
00337
00338     /* Lock in 50% duty for clean scope capture. No sweep. */
00339     REG32(GTCCRA) = (uint32_t)(k_pwm_period_cnt / 2U);
00340     REG32(GTCCRB) = (uint32_t)(k_pwm_period_cnt - (k_pwm_period_cnt / 2U));
00341
00342     /* PA7 heartbeat @ slow rate so we can see firmware is alive. */
00343     REG8(PORTA_PDR) |= PA7_MASK;
00344     for (;;) {
00345         REG8(PORTA_PODR) ^= PA7_MASK;
00346         for (volatile uint32_t d = 0; d < 500000U; d++) { __asm__ volatile("nop"); }
00347     }
00348     return 0;
00349 }
00350
00351 /*
=====
00352 * Stubs for symbols referenced by vectors.S (ThreadX ISRs, CMT0) that this
00353 * minimal firmware doesn't use. Prevents linker errors without dragging in
00354 * the full ThreadX library or a real CMT0 driver.
00355 *
=====
*/
00356 void __tx_thread_context_save(void) {}
00357 void __tx_thread_context_restore(void) {}
00358 void cmt0_isr(void) {}

```

